

MoZuMa: A Model Zoo for Multimedia Applications

Stéphane Massonnet
EPFL
stephane.massonnet@epfl.ch

Marco Romanelli
EPFL
marco.romanelli@epfl.ch

Rémi Lebreton
EPFL
remi.lebreton@epfl.ch

Niels Poulsen
EPFL
niels.poulsen@epfl.ch

Karl Aberer
EPFL
karl.aberer@epfl.ch

ABSTRACT

Lots of machine learning models with applications in Multimedia Search are released as Open Source Software. However, integrating these models into an application is not always an easy task due to the lack of a consistent interface to run, train or distribute models. With MoZuMa, we aim at reducing this effort by providing a model zoo for image similarity, text-to-image retrieval, face recognition, object similarity search, video key-frames detection and multilingual text search implemented in a generic interface with a modular architecture. The code is released as Open Source Software at <https://github.com/mozuma/mozuma>.

CCS CONCEPTS

• **Computing methodologies** → **Machine learning**; Computer vision; Natural language processing.

KEYWORDS

Vision and Language, Multimedia Search, Open Source Software

ACM Reference Format:

Stéphane Massonnet, Marco Romanelli, Rémi Lebreton, Niels Poulsen, and Karl Aberer. 2022. MoZuMa: A Model Zoo for Multimedia Applications. In *Proceedings of the 30th ACM International Conference on Multimedia (MM '22)*, October 10–14, 2022, Lisboa, Portugal. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3503161.3548542>

1 INTRODUCTION

When looking at extracting information from images, videos or text documents, many pre-trained models such as OpenAI CLIP [7], ArcFace [2] or VinVL [10] are released as Open Source Software. These models, often available in a deep learning framework, implement algorithms including, but not limited to, text-to-image retrieval [7], face recognition [2, 6, 9], object detection [10] or semantic text search [8]. Complex search engines can be built when integrating these models together by combining queries on different aspects of an image, a video or a text document. For instance, one could think about searching for a specific object in images from tweets related to a topic of interest. Such a query would combine an object

detection model with a semantic text search model. Combining the output of these models quickly becomes an engineering challenge as they are released by different research teams with very different conventions.

The first issue one can encounter is the installation procedure. Some models come from large repositories hosting many different models for research purposes. While these repositories are great resources for machine learning researchers, they can come with many dependencies making the installation overly complex when interested in a couple of models. After passing the installation step, one should retrieve the model state. The repositories used to store model weights are diverse and some of them require manual downloads from OneDrive¹ or Google Drive². This is not suitable when building an application that requires access to these states at any time. Therefore, an engineer building an application would have to manually copy the model states and change the internal code to load it from a different place. Once the state is loaded, the model can be executed. But again, due to the lack of standardisation, the returned data for features, labels or bounding boxes will differ and extra efforts will have to be put to normalise these predictions. The whole process has to be repeated when new state-of-the-art models are released or when a new type of model has to be integrated.

In order to rationalise this integration effort, we developed a Python library called MoZuMa, it provides models for information extraction from images, videos or text documents implemented with a generic interface. It aims at supporting engineers and researchers when using Open Source pre-trained models by reducing the need for complex installation and integration. This library brings:

- (1) a generic interface to train, run and manage internal state of models with initial support for PyTorch;
- (2) a collection of ready-to-use models for image, video and text information extraction;
- (3) a modular architecture to easily add new features.

It is released as Open Source Software³ under an MIT Licence. MoZuMa is not meant to be a replacement for existing machine learning frameworks. It is rather an abstraction layer on top of them to make it easy to switch between models or backends without having to change the integration code. Also, although as of today MoZuMa supports PyTorch models, its APIs are not specific to PyTorch and could run models from other frameworks as well.

In the following sections, we will see an overview of the generic interface introduced by MoZuMa, the main components of its architecture, a list of applications of the model zoo, an example of an

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
MM '22, October 10–14, 2022, Lisboa, Portugal

© 2022 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-9203-7/22/10...\$15.00
<https://doi.org/10.1145/3503161.3548542>

¹<https://onedrive.live.com/>

²<https://drive.google.com/>

³<https://github.com/mozuma/mozuma>

integration of MoZuMa into a web application and a discussion about future work.

2 API OVERVIEW

MoZuMa exposes an API that normalises how to run model inference or training. In the following example, we'll load a PyTorch implementation of a ResNet [4] model pre-trained on ImageNet [1]. Implementing this model is done by calling a model factory for ResNets pre-trained on ImageNet with the desired architecture variant as parameter.

```
model = torch_resnet_imagenet("resnet18")
```

For this model, we are interested at collecting features and image classification labels in-memory. This can be done by defining callbacks. Callbacks are used to add custom logic while the model is being run.

```
callbacks = [
    CollectFeaturesInMemory(),
    CollectLabelsInMemory(),
]
```

The model takes a dataset of images that we construct by passing a list of local filenames.

```
dataset = ImageDataset(
    LocalBinaryFilesDataset(list_of_filenames)
)
```

Finally, we can define and run a runner object. In this scenario, we are using a runner that will execute a single-GPU inference of our model against a dataset of images calling the provided callbacks to handle the features and labels.

```
runner = TorchInferenceRunner(
    model=model,
    dataset=dataset,
    callbacks=callbacks,
    options=TorchRunnerOptions()
)
runner.run()
```

Note that, in the code above, the runner can be modified with a runner doing the inference with multiple GPUs by simply replacing the runner class. Similarly, we could run another PyTorch model by swapping the model argument. This highlights the modularity of MoZuMa's APIs. The integration into an application can be done once and models, datasets, callbacks or runners can easily be replaced with a change of configuration.

3 ARCHITECTURE

MoZuMa is implemented around five main components: models, datasets, runners, callbacks and stores. See Figure 1, for a high-level description of the interactions between these components. The architecture is meant to be modular and new functionalities can be added by implementing and replacing components. For instance, changing the way results of inference (e.g. features, labels, etc.) are handled can be done by implementing a custom callback class, which will be required when integrating MoZuMa into an application storing data in a database.

The following sections will go through the description and purpose of all the main components of MoZuMa.

3.1 Models

The model contains the code to learn or extract information from an input. In the case of PyTorch, the model is a PyTorch module.

In terms of APIs, the model implementation has to comply with the requirements of the runner it is used with. Additionally, if using MoZuMa's stores, the model will also have to implement additional methods to get and set its state. Other than that, there is no restriction on how a model should be implemented.

3.2 Datasets

Datasets implement a collection of data. Again, the implementation of a dataset depends on the requirements of the runner used. As of today, there is no restriction in MoZuMa on how a dataset should be implemented.

For instance, in the case of PyTorch, all the available runners expect to receive a class derived from the PyTorch datasets implementation.

3.3 Runners

A runner is responsible to run inference or train a model against a dataset. It takes four objects as parameters: a model, a dataset, callbacks and options. Each runner will impose type constraints on these arguments. For instance, the runners for PyTorch will expect a model to implement a PyTorch module.

The runner plays an orchestrator role, it fetches data from the dataset, runs the model against this data and sends results to callbacks.

3.4 Callbacks

Callbacks are used to handle data produced by a runner. For instance, a runner that runs inference on a model will use callbacks to save features, labels, etc. On the other hand, a runner that does model training will use callbacks to save model checkpoints and final states.

3.5 Stores

Stores handle loading and saving model states as binary files. For a model to be compatible with stores, it must implement three methods `state_type`, `get_state` and `set_state`. A state type is a unique identifier for a model's state structure. It is used by the store to identify which state can be loaded in a model. It combines information from the framework and the model architecture. A PyTorch ResNet-18 for ImageNet classification will have a state type combining `pytorch`, `resnet18` and `cls1000` (for the 1000 classes of ImageNet).

MoZuMa provides implementations of multiple state stores to save model weights on local disk, S3 object storage or in GitHub releases.

4 APPLICATIONS

MoZuMa contains a model zoo (Table 1) that can be used to extract information from images, videos or text. As of today, these models have been picked to support complex retrieval of images, videos or

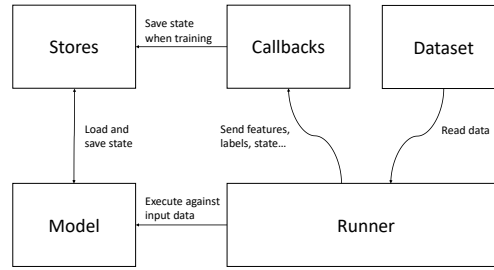


Figure 1: Architecture overview of the main components of MoZuMa

Table 1: MoZuMa’s model zoo

Category	Models
Image classification	ResNet [4], DenseNet [5]
Face detection	MTCNN [9]
Face recognition	ArcFace [2], MagFace [6]
Object detection	VinVL [10]
Video key-frames	Custom approach (see Section 4.6)
Sentence embedding	DistilUseMultilingualCasedV2 [8]

text in the context of large collections of data crawled online. Therefore, the following section focuses on this use-case but MoZuMa’s APIs are not limited to the applications listed below.

4.1 Image similarity search

Searching for similar images is a natural way of filtering large collections from a query image. With MoZuMa, this can be done by computing features and comparing them with cosine similarity. These features can be obtained using the provided ResNet [4] and DenseNet [5] models pre-trained on ImageNet [1] or Places365 [11]. ImageNet pre-training retrieves images with similar main subjects while Places365 pre-training will focus on images with similar locations.

4.2 Text-to-image retrieval

As sometimes a good query image is not available, it can be interesting to query collections directly from text. For this type of queries, MoZuMa provides the CLIP [7] image and text encoders. These encoders provide features that share the same space across text and images. This makes similarity search between features of text and features of images possible with cosine similarity.

The implementation of this text-to-image retrieval should be done in two phases. First, an offline process computes features for all images using the image encoder. Then, at query time, the text query is encoded with the text encoder and images are ranked with cosine similarity between the text query features and the image collection features.

4.3 Face recognition

When dealing with images, another interesting aspect is the ability to search for faces. Face search is done in two steps (1) face detection

to detect where the faces are in the image and (2) face recognition to quantify faces similarity. MoZuMa provides models for face detection (MTCNN [9]) and for face recognition (ArcFace [2], MagFace [6]). This allows, from a query face, to find all similar faces in other images.

4.4 Object similarity search

MoZuMa provides an object detection model extracted from VinVL’s object detector [10]. This model can detect a wide variety of objects with 1594 classes. However, while it can be used directly with object labels, we find it interesting to use it to search for similar objects. In MoZuMa’s implementation, this model returns features for objects. This makes it possible to pick an object from a candidate image and retrieve all other images containing similar objects. Once again, we use cosine similarity of features to achieve this.

This is especially useful when searching for objects attributes that are not part of the model’s predicted classes. For instance, when searching for a specific type of outfit, vehicle or flag.

4.5 Custom image tagging

Sometimes, it is not possible to get the right filter with just similarity search. For instance, we might want to add a label to faces representing emotions or to flag images containing a specific object.

For these scenarios, MoZuMa provides training runners that can re-train a whole model. But we also provide classification heads that can be used on top of existing features. The classification heads being small, training them is fast. This can be a quick and effective solution to add custom labels on images.

4.6 Video key-frames extraction

Out of all data collected online, there is a significant share of videos. These videos are sequences of images so technically all the applications listed above could be applied to videos. Unfortunately, with 24 frames per second, running models against all frames would be inefficient.

We thus developed in MoZuMa an algorithm for an efficient video key-frame extraction. It turns a video into a sequence of frames with the objective of only keeping unique frames, ie. removing redundant frames. Each video frame is first encoded using one of the available image encoder models in MoZuMa. We usually select a small ResNet-18 [4] for faster execution. Unique frames are then selected by clustering the encoded frames and keeping only the medoids of each cluster. We cluster the frames using k-means algorithm, where

the optimal number of clusters k is chosen using the silhouette method.

This drastically reduces the number of images other models have to process while keeping a representative set of frames for the video. Using this algorithm, image similarity, face recognition, object similarity search and custom image tagging can be used on videos as well.

4.7 Multilingual text search

As the data we collect often comes from Twitter, we have identified the need to have a model to support multilingual text search. MoZuMa provides DistilUseMultilingualCasedV2 [8], a multilingual sentence embedding model. This model can be used to extract features from documents and search for similar documents using cosine similarity.

5 DEMONSTRATION

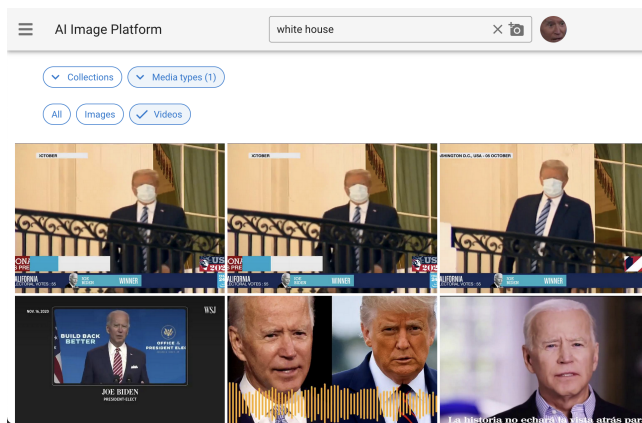


Figure 2: Example of the integration of MoZuMa in a web application. It shows a working prototype of a combined text-to-image and face search query on videos.

MoZuMa is being used to power a web application shown in Figure 2. The purpose of this application is to create searchable indexes for images or videos collected from the web (mainly from Twitter). It uses MoZuMa as a backend and covers most of the features listed in Section 4.

For instance, it integrates with text-to-image retrieval to power an image search. In Figure 2, the searched text is `white house`. This will order results based on similarity between the encoded text and encoded images, returning images related to the `white house` first.

It also supports filtering from similar images, faces or objects. For instance, in Figure 2, there is an active face filter for Joe Biden. The same type of filter can be used for objects.

6 FUTURE WORK

We are looking at adding new models as they are released to maintain state-of-the-art performance of MoZuMa’s model zoo on the applications listed in Section 4. We will also be adding models to support more modalities. For instance, speech recognition models to be able to search in video’s audio tracks.

As discussed in Section 3, MoZuMa is not tight to a specific machine learning framework even though this version only contains PyTorch models. In the future, we will be looking at supporting other runtimes such as ONNX [3]. This would allow for lightweight deployment with better performance.

Finally, we believe that the current architecture can support online inference for applications receiving a stream of messages (such as the Twitter stream API). We are looking at implementing a combination of runner, datasets and callbacks to support this scenario.

7 CONCLUSION

In this work, we introduced MoZuMa a Python library that proposes a generic interface to run machine learning models with a collection of pre-trained models for image similarity, text-to-image retrieval, face recognition, object detection, video key-frames extraction and multilingual text search. We have shown that this library has been designed to be modular and extensible to facilitate its integration into an application. We’ll continue to maintain it with the goal to keep the model zoo up-to-date and propose support for more runtimes in addition to PyTorch.

REFERENCES

- [1] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. 2009. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*. Ieee, 248–255.
- [2] Jiankang Deng, Jia Guo, Xue Niannan, and Stefanos Zafeiriou. 2019. ArcFace: Additive Angular Margin Loss for Deep Face Recognition. In *CVPR*.
- [3] ONNX Runtime developers. 2021. ONNX Runtime. <https://onnxruntime.ai/>. Version: x.y.z.
- [4] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep Residual Learning for Image Recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 770–778. <https://doi.org/10.1109/CVPR.2016.90>
- [5] Gao Huang, Zhuang Liu, Laurens van der Maaten, and Kilian Q. Weinberger. 2017. Densely Connected Convolutional Networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.
- [6] Qiang Meng, Shichao Zhao, Zhida Huang, and Feng Zhou. 2021. MagFace: A Universal Representation for Face Recognition and Quality Assessment. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 14225–14234.
- [7] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. 2021. Learning Transferable Visual Models From Natural Language Supervision. In *Proceedings of the 38th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 139)*, Marina Meila and Tong Zhang (Eds.). PMLR, 8748–8763. <https://proceedings.mlr.press/v139/radford21a.html>
- [8] Nils Reimers and Iryna Gurevych. 2020. Making Monolingual Sentence Embeddings Multilingual using Knowledge Distillation. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics. <https://arxiv.org/abs/2004.09813>
- [9] K. Zhang, Z. Zhang, Z. Li, and Y. Qiao. 2016. Joint Face Detection and Alignment Using Multitask Cascaded Convolutional Networks. *IEEE Signal Processing Letters* 23, 10 (Oct 2016), 1499–1503. <https://doi.org/10.1109/LSP.2016.2603342>
- [10] Pengchuan Zhang, Xiujun Li, Xiaowei Hu, Jianwei Yang, Lei Zhang, Lijuan Wang, Yejin Choi, and Jianfeng Gao. 2021. VinVL: Revisiting Visual Representations in Vision-Language Models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 5579–5588.
- [11] Bolei Zhou, Agata Lapedriza, Aditya Khosla, Aude Oliva, and Antonio Torralba. 2017. Places: A 10 million Image Database for Scene Recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2017).